

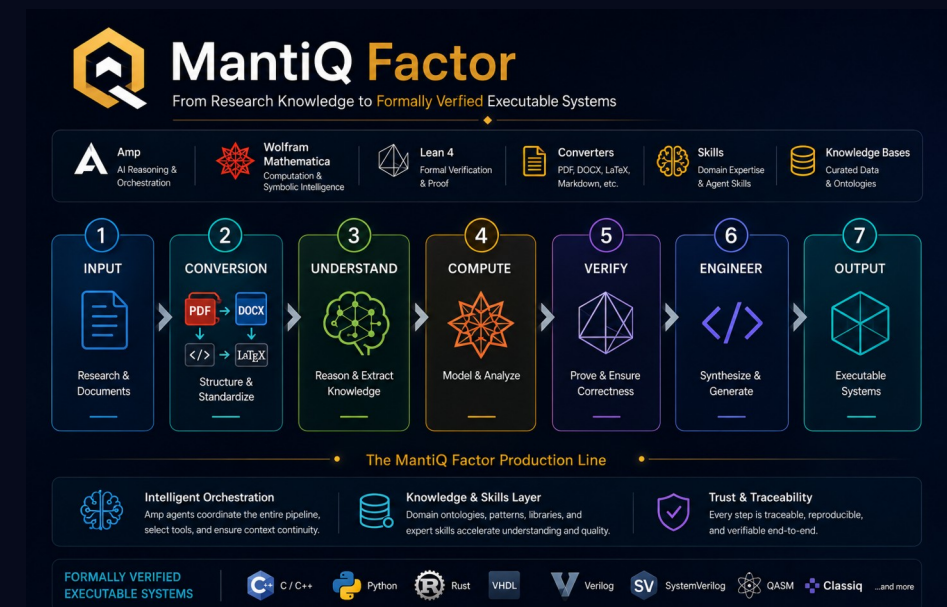
From a PDF brief to a verified quantum program

Antenna tilt optimization with QAOA on real Tel Aviv cell-tower data, computed in Wolfram, proved in Lean 4, engineered and run on Classiq

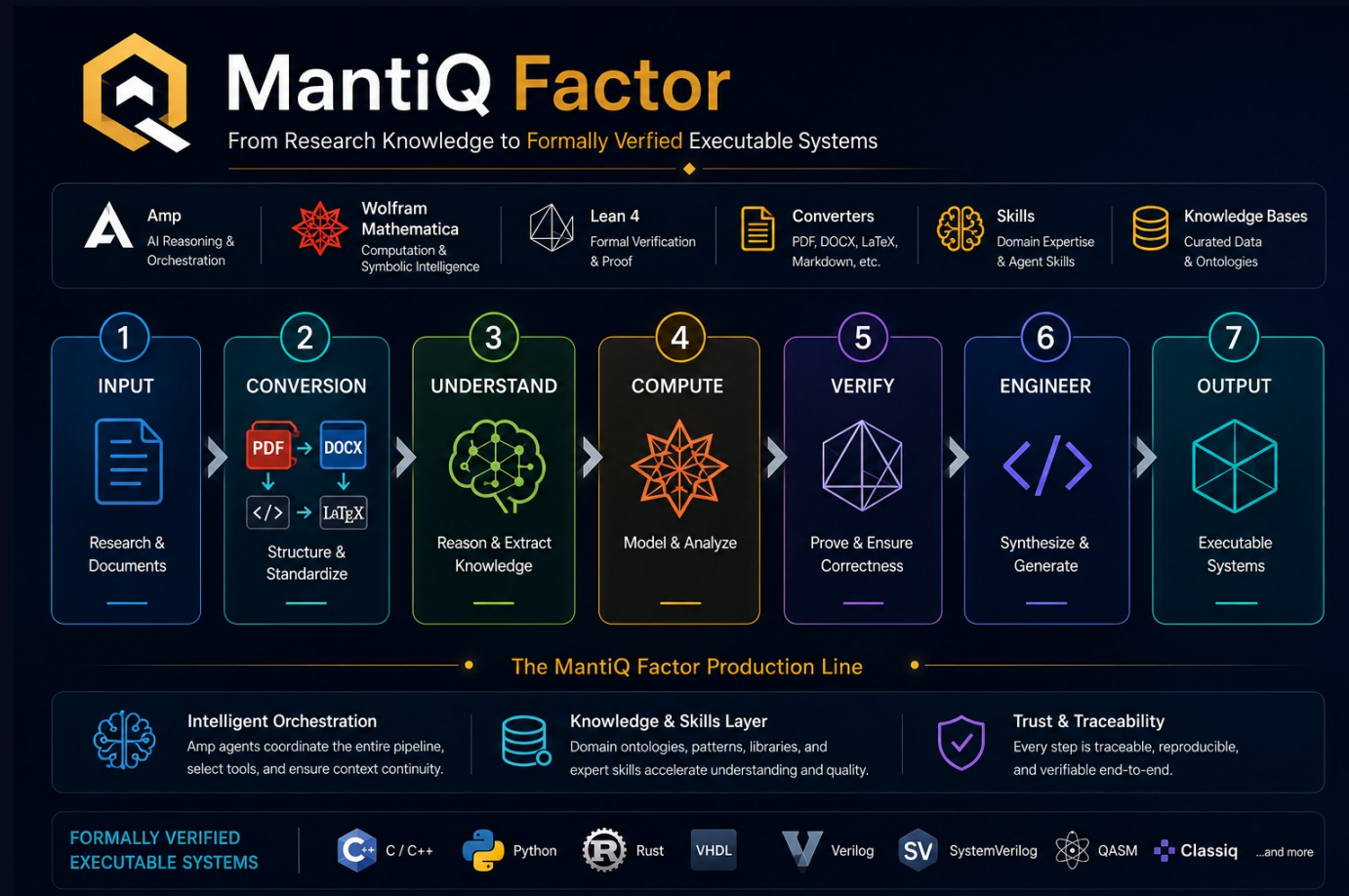
Daniel Liezrowice

Team Quantum Headache, QUBIT 2026 hackathon | September 2026

github.com/zuwasi/mantiq-factor-antenna-tilt



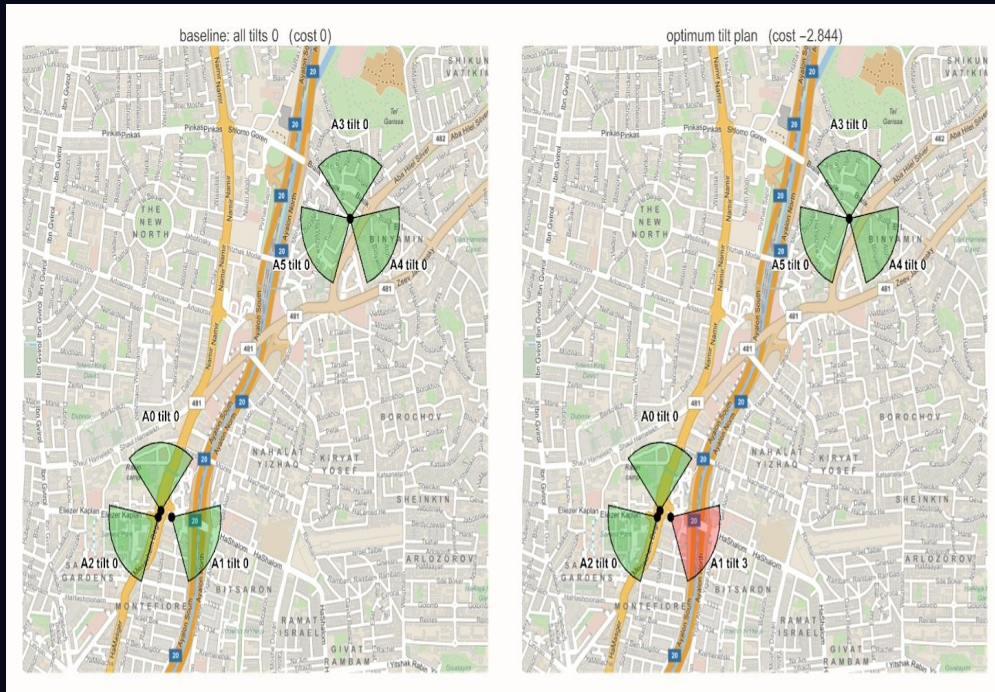
MantiQ Factor: a seven-stage production line



The pipeline: Input, Conversion, Understand, Compute, Verify, Engineer, Output. Tools: Amp (agentic orchestration), Wolfram Mathematica, Lean 4, converters, skills, knowledge bases.

- ▶ Every stage leaves an artifact on disk (Markdown, JSON, .nb, .lean, .qmod) so the next stage consumes a file, not a memory
- ▶ Compute and Engineer are separated: the exact simulation picks the angles, the SDK builds the circuit
- ▶ Verify is a formal step, not a code review: Lean 4 theorems about the cost function and the circuit
- ▶ One agent (Amp) drives the tools; humans decide the model and read the numbers
- ▶ Result: every claim in the final program traces back to a proof, a simulation or a dataset

Stage 1, Input: the challenge and what the team handed over



Six real sector antennas from OpenCellID, Tel Aviv (data/opencellid_il2.csv). Interference matrix from pairwise distances and azimuths.

- ▶ Challenge (QUBIT 2026): choose the electrical down-tilt of every sector antenna. Tilting down cuts interference on neighbours but shrinks coverage
- ▶ Inputs received as documents, not code: the challenge brief (PDF), the team Hamiltonian note (Markdown with LaTeX), the k-means hierarchy concept (PDF), the group deck (Google Slides)
- ▶ Team utility: $U(t) = \alpha Q(t) - \beta I(t) - \gamma D(t)$; quality, interference, unmet demand
- ▶ Team proposal: 27 three-sector configurations packed into 5 qubits, groups by k-means with $k = 5$
- ▶ Constraint: Classiq student edition, 15 parallel jobs, simulator only

Stage 2, Conversion: documents become machine-readable facts

PDF brief, team PDF

Read with a PDF converter into Markdown; formulas kept as LaTeX. Output:
docs/qaoa_antenna_hamiltonian_explanation.md

Google Slides deck

Exported to PDF, read slide by slide, merged into one PPTX built from code (build_deck.js), so the deck is reproducible too

OpenCellID CSV

Pipeline data/opencellid_pipeline.py: filter Tel Aviv sectors, compute distances, azimuth overlap, interference matrix I and coverage vector c

What conversion decided (and recorded in the Markdown):

- ▶ Keep the team utility form; maximize U is the same as minimize Cost = $-U$ (later proved as theorem maximize_utility_iff_minimize_negative)
- ▶ Fix the weights for the real dataset: $\beta = 1$ (interference $t^T I t$), $\alpha = 1.5$ (coverage $c \cdot t$), $\gamma = 0.3$ (tilt penalty sum t)
- ▶ Four discrete tilt levels per antenna (0, 2, 4, 6 degrees) instead of continuous angles: the register is exactly 2 qubits
- ▶ Six antennas give $4^6 = 4096$ plans: small enough to enumerate exactly, so every quantum number can be checked against the truth

Stage 3, Understand: 2 qubits per antenna, not a 5-qubit pack

	Team proposal: 27 configs in 5 qubits	Chosen: 2 qubits per antenna (QNum[2])
Encoding	3 tilts x 3 sectors = 27 states of 32; 5 states invalid	4 tilts per antenna; all 4096 states valid
Cost Hamiltonian	Arbitrary 5-qubit diagonal: terms up to ZZZZZ	2-local Ising: 12 Z + 60 ZZ + offset (73 terms)
Invalid-state mass (team numbers)	15.6% at start, 3% after training	0%, by construction
P(optimum) / uniform (team numbers)	0.5x to 1.2x	27x (Wolfram), 27x to 29x (Classiq)
Scales to more antennas	No: 3 sectors fixed per pack	Yes: 2n qubits, still 2-local
Provable	Hard: decode is a lookup table	decodeTilt is a bijection (Lean theorem)

- ▶ This is the decision the whole pipeline depends on. It was made here, with the exact enumeration in hand, before any circuit existed
- ▶ The team hierarchy idea (k-means groups, one QAOA per group, recursive refinement) was kept: our 12-qubit problem is one leaf of that tree

Stage 3, Understand: the cost function and its Pauli decomposition

Cost(t) = t^T I t - 1.5 c . t + 0.3 sum(t)

t in {0,1,2,3}⁶ (tilt levels), I = 6x6 interference matrix from the map, c = coverage per antenna
tilt_i = 2 q_{2i} + q_{2i+1}, so t_i is linear in its two qubits and t_i t_j gives only ZZ terms

Pauli weight	terms	meaning
0	1	offset 35.499 (global phase, dropped)
1 (Z)	12	coverage gain and tilt penalty, 2 per antenna
2 (ZZ)	60	15 interfering pairs x 4 qubit pairs
3, 4, 5	0	none: the model is 2-local

gates/pauli_terms.json: Walsh-Hadamard transform of the 4096-entry cost table, checked to 1e-9 against every entry.

4096

plans enumerated exactly

-2.844

unique minimum at (0,3,0,0,0,0)

73

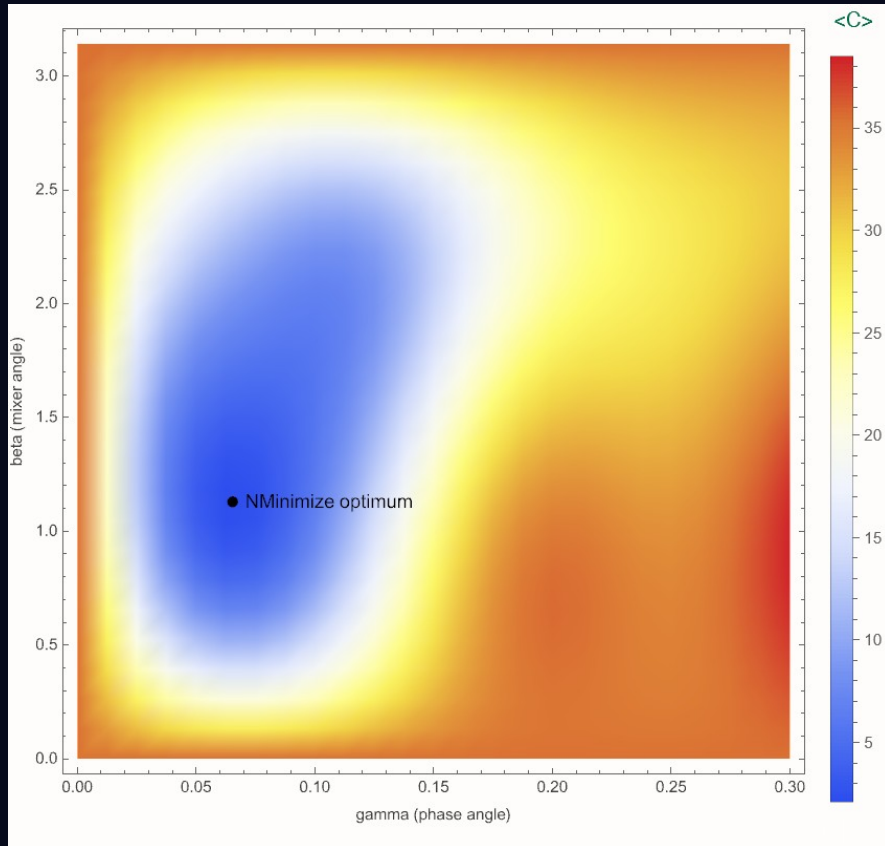
Pauli terms, all weight <= 2

12

qubits: 6 antennas x QNum[2]

- Only antenna 1 tilts (level 3); the other five stay flat. That is physically sensible: it is the antenna with the most overlap
- Naive layer cost: 72 RZ + 120 CX; Classiq synthesis merges parity ladders down to 92 CX

Stage 4, Compute: Wolfram exact simulation picks the angles



Expected cost over (γ, β) for $p = 1$, evaluated exactly on the 4096-dimensional state. Minimum marked.

- ▶ wolfram/wolfram_realdataset_algorithms.wl builds H_C from the same JSON, applies $\exp(-i \gamma H_C)$ and the RX mixer as dense matrices, no sampling noise
- ▶ $p = 1$ optimum: $\gamma = 0.0654$, $\beta = 1.1293$, giving $\langle \text{Cost} \rangle = 1.9985$, $P(\text{cost} < 0) = 0.53$, $P(\text{optimum}) = 0.0066$ (27x uniform)
- ▶ $p = 2$ optimum: $\langle \text{Cost} \rangle = -0.3825$, $P(\text{cost} < 0) = 0.67$
- ▶ Also compared: VQE, Grover-style amplification and plain uniform sampling on the same cost table
- ▶ The angles are written to wolfram_realdataset_algorithms.json: this is the hot start the Classiq circuit later uses

1.9985

predicted $\langle \text{Cost} \rangle$, $p=1$

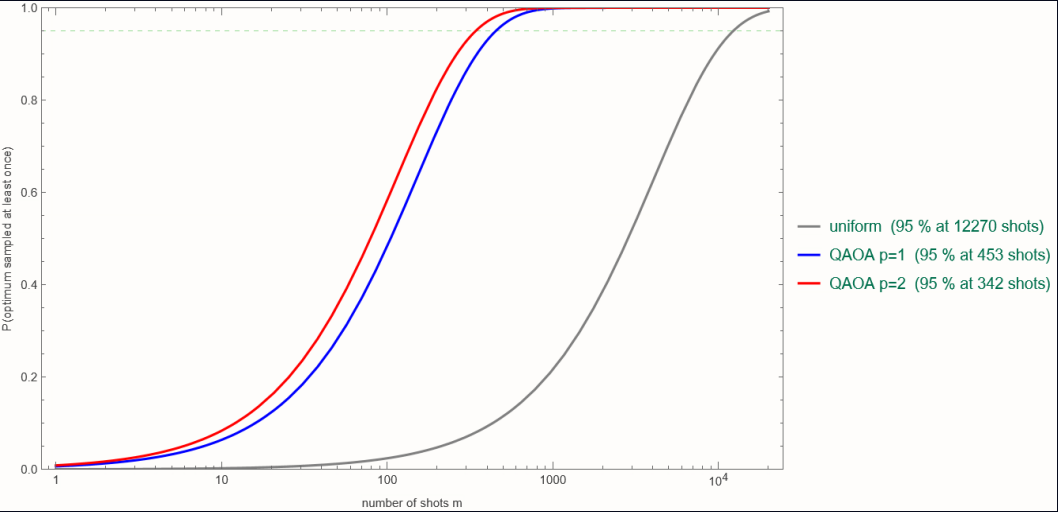
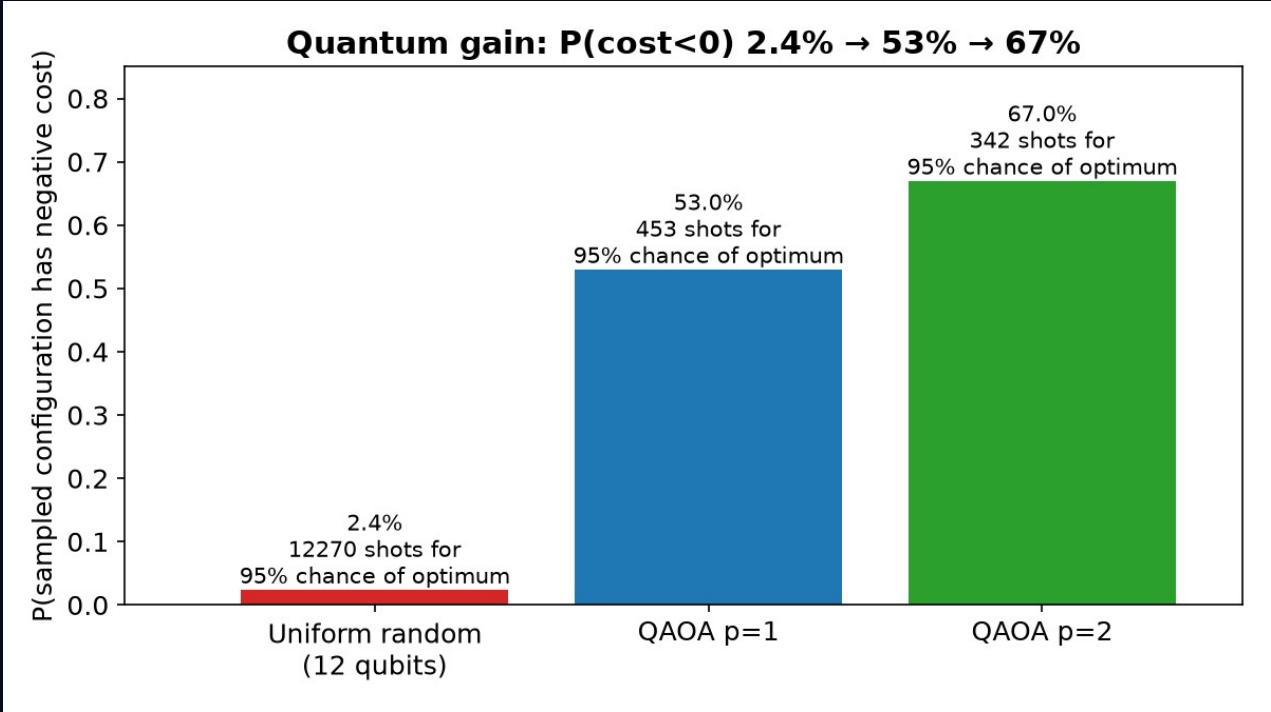
0.53

$P(\text{cost} < 0)$, $p=1$

27x

$P(\text{optimum})$ vs uniform

Stage 4, Compute: what the quantum gain is worth in shots



charts/quantum_gain.png: probability of the optimum and of a negative-cost plan, uniform vs QAOA p=1 and p=2.

- Honest framing: at 12 qubits a laptop enumerates 4096 plans instantly. The gain is a property of the algorithm we can measure exactly here and carry to sizes where enumeration stops (slide 13).

sampler	P(opt)	shots for 95%
uniform	1/4096	12,270
QAOA p = 1	0.0066	about 450
QAOA p = 2	0.0098	about 300

Stage 5, Verify: Lean 4 theorems, zero sorry

```
def decodeTilt (q0 q1 : Fin 2) : Fin 4 := ...
theorem decodeTilt_bijective :
  Function.Bijective (fun q : Fin 2 x Fin 2 => decodeTilt q.1 q.2)

def costS (t : Fin 6 -> Fin 4) : Int := ... -- scaled x 10^4
def optTilt : Fin 6 -> Fin 4 := ![0, 3, 0, 0, 0, 0]
theorem costS_optTilt : costS optTilt = -28440
theorem costS_lower_bound : forall t, -28440 <= costS t
theorem costS_unique : forall t, costS t = -28440 -> t = optTilt

theorem cost_phase_norm : forall gamma c : Real,
  norm (Complex.exp ((gamma * c : Real) * Complex.I)) = 1
theorem cost_phase_preserves_probability : ...
theorem two_smul_P0 : (2 : Rat) * P0 = 1 + Z -- Z decomposition
```

- ▶ Encoding: the 2-qubit register maps one-to-one onto the four tilt levels, so no state is invalid or double-counted
- ▶ Classical truth: -2.844 is the minimum, it is attained at (0,3,0,0,0,0), and nowhere else
- ▶ Circuit: the cost layer $\exp(i \text{ gamma Cost})$ has unit norm, so it changes phases only and keeps every measurement probability of a basis state
- ▶ Bridge to the gate set: $2 P0 = 1 + Z$ and $2 P1 = 1 - Z$ justify the Z-only decomposition of the Hamiltonian
- ▶ These are the statements the Wolfram numbers and the Classiq samples are later checked against

lean/TiltQAOA.lean, 109 lines, Mathlib. Build log: lean/tilt_qaoa_lean_build.log. Cost scaled by 10^4 so the theorems are over integers and decide by computation.

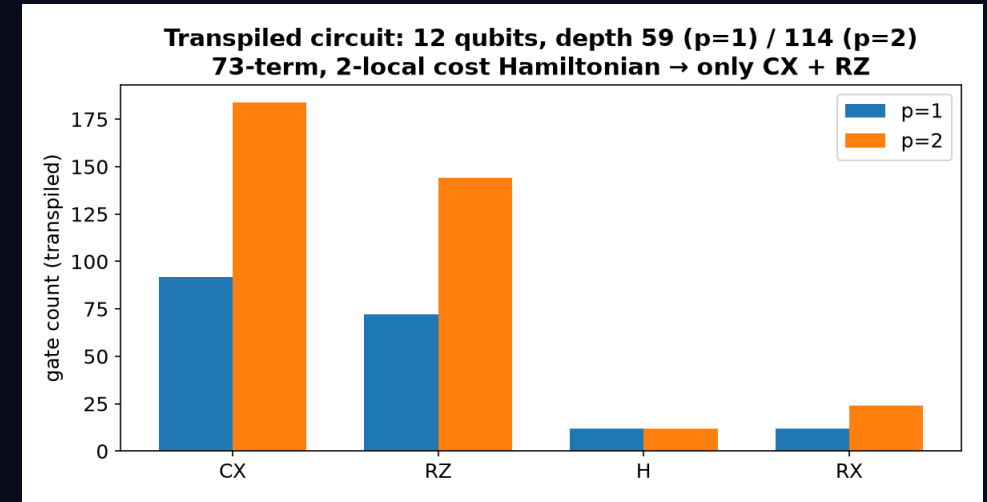
Stage 6, Engineer: the cost becomes a Qmod phase() statement

```
class Tilts(QStruct):
    tilts: QArray[QNum[2], 6]          # 6 antennas x 2 qubits

def cost(t: Tilts) -> CReal:
    interference = sum(I[i][j] * t.tilts[i] * t.tilts[j] ...)
    coverage     = sum(c[i] * t.tilts[i] ...)
    return interference - 1.5 * coverage + 0.3 * sum(t.tilts)

@qfunc
def main(gammas: CArray[CReal, P], betas: CArray[CReal, P],
         t: Output[Tilts]):
    allocate(t)
    hadamard_transform(t)
    repeat(P, lambda k: [phase(cost(t), gammas[k]),
                        apply_to_all(lambda q: RX(2*betas[k], q), t)])

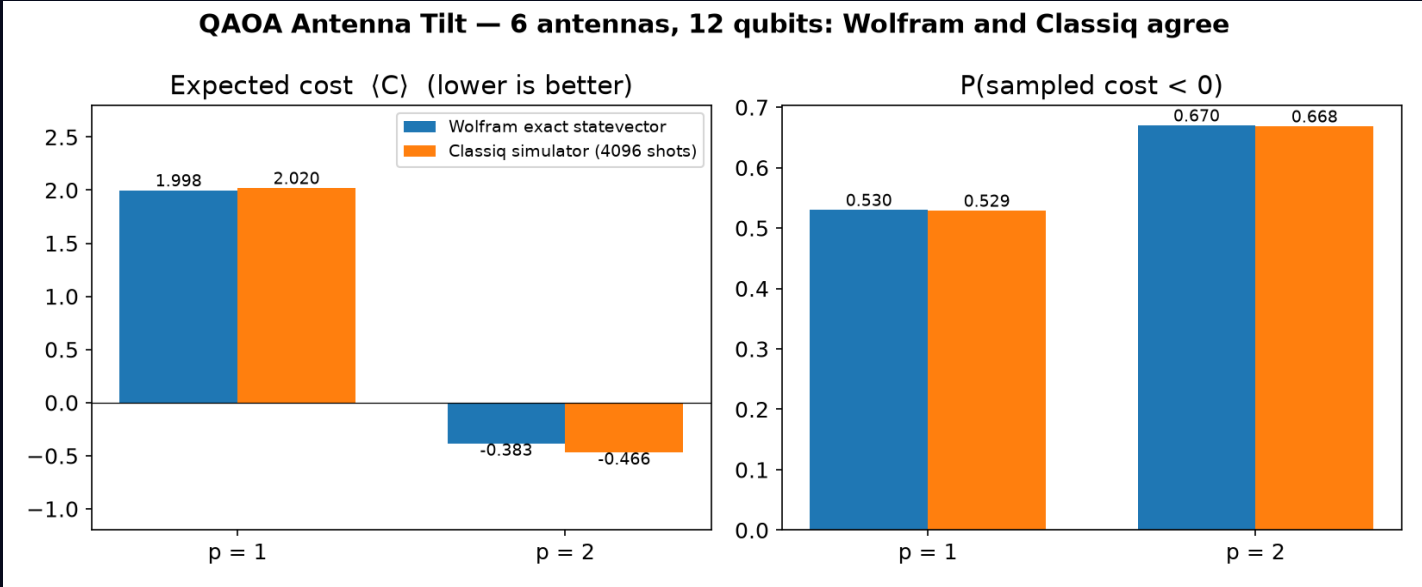
qprog = synthesize(qmod)  # width 12, depth 59, 92 CX
res = execute(qprog, gammas=[0.0654], betas=[1.1293])  # hot start
```



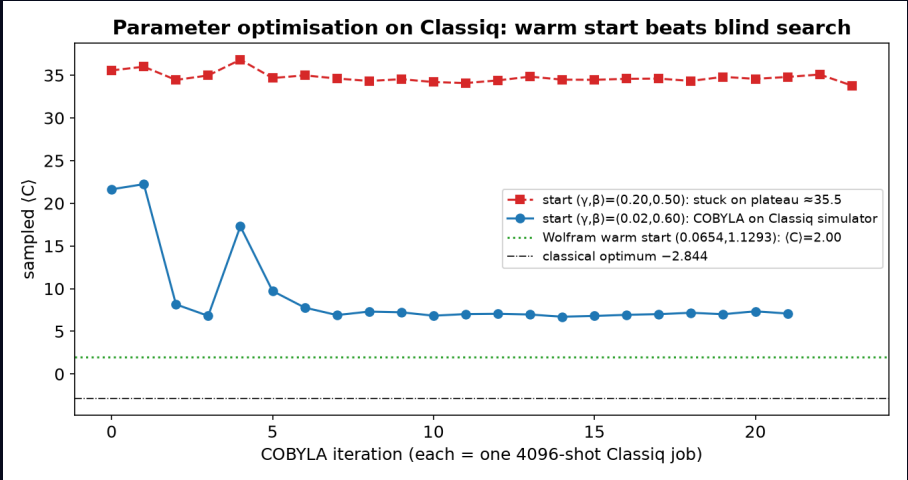
charts/circuit_stats.png: synthesized circuit sizes for p = 1 and p = 2.

- ▶ No hand-built CX ladders: phase() takes the arithmetic cost expression and Classiq synthesizes the diagonal unitary
- ▶ The Pauli decomposition (gates/) exists only to predict gate counts and to feed Wolfram; the SDK never sees it
- ▶ Deliverables:
antenna_tilt_optimization.ipynb, .qmod, .synthesis_options.json, .metadata.json (classiq-library format)
- ▶ classiq/qaqa_tilt_classiq.py is the standalone script with COBYLA on the simulator

Stage 7, Output: Classiq reproduces the Wolfram prediction



charts/wolfram_vs_classiq.png: exact statevector (Wolfram) against 4096-shot samples on the Classiq simulator at the same angles.



COBYLA on Classiq starting from the Wolfram angles: 8 iterations, no improvement over the hot start, angles stay at $(0.065, 1.149)$.

2.06

Classiq $\langle \text{Cost} \rangle$ $p=1$ (pred. 2.00)

0.525

$P(\text{cost} < 0)$ (pred. 0.53)

29x

$P(\text{optimum})$ vs uniform (pred. 27x)

-0.39

Classiq $\langle \text{Cost} \rangle$ $p=2$ (pred. -0.38)

(0,3,0,0,0,0)

best sampled plan = proven optimum

Three independent engines, one set of numbers

quantity	Lean 4 (proof)	Wolfram (exact state)	Classiq (4096 shots)	agree
minimum cost	-2.8440 (theorem)	-2.8440	-2.8440 (best sample)	yes
argmin	(0,3,0,0,0,0), unique	(0,3,0,0,0,0)	(0,3,0,0,0,0)	yes
encoding valid	bijection	4096 states, all decode	all samples decode	yes
<Cost> p=1 at hot start		1.9985 (Qiskit statevector of the exported QASM: 1.9985)	2.02 to 2.06 (4096 shots), 1.99 after COBYLA	shot noise
P(cost < 0) p=1		0.53	0.525	yes
P(optimum) p=1		0.0066 (27x)	0.0066 to 0.0071 (27x to 29x)	yes
<Cost> p=2		-0.3825	-0.39	yes
phase layer diagonal, unit norm	theorem	matrix check	synthesized as RZ ladders only	yes

- ▶ The value of MantiQ Factor is this table: no number in the demo rests on a single tool
- ▶ Blind COBYLA starts on Classiq (hackathon runs): start (0.20, 0.50) stuck at <Cost> 35.5, start (0.02, 0.60) reached only 7.0 after 20 jobs. From the Wolfram angles: 2.0 in the first job, and 8 more iterations found nothing better

Where this leaf sits: k-means hierarchy and the road to 1000 qubits

10³⁰¹

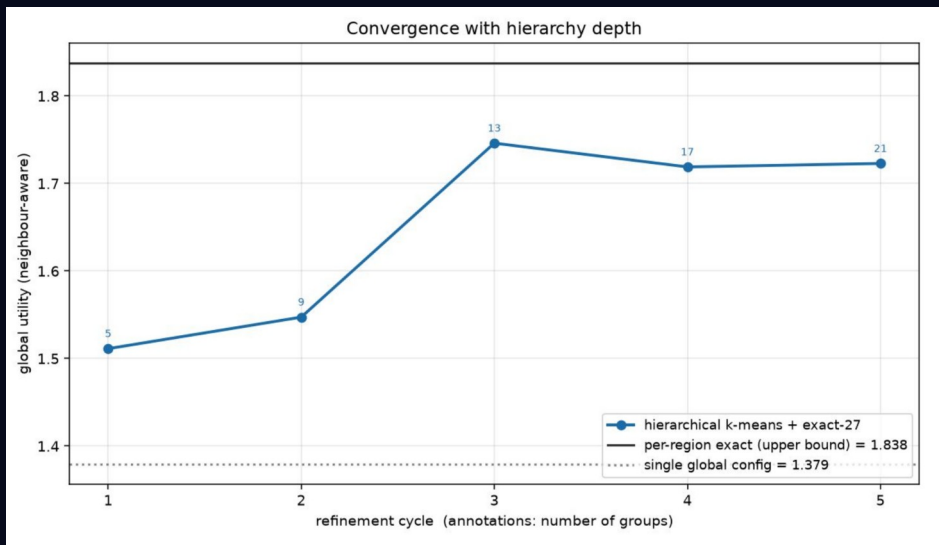
plans for 500 antennas x 4 tilts (today: 4096)

about 45 qubits

ceiling of exact statevector simulation (50 q = 18 PB)

about 13,000 CX

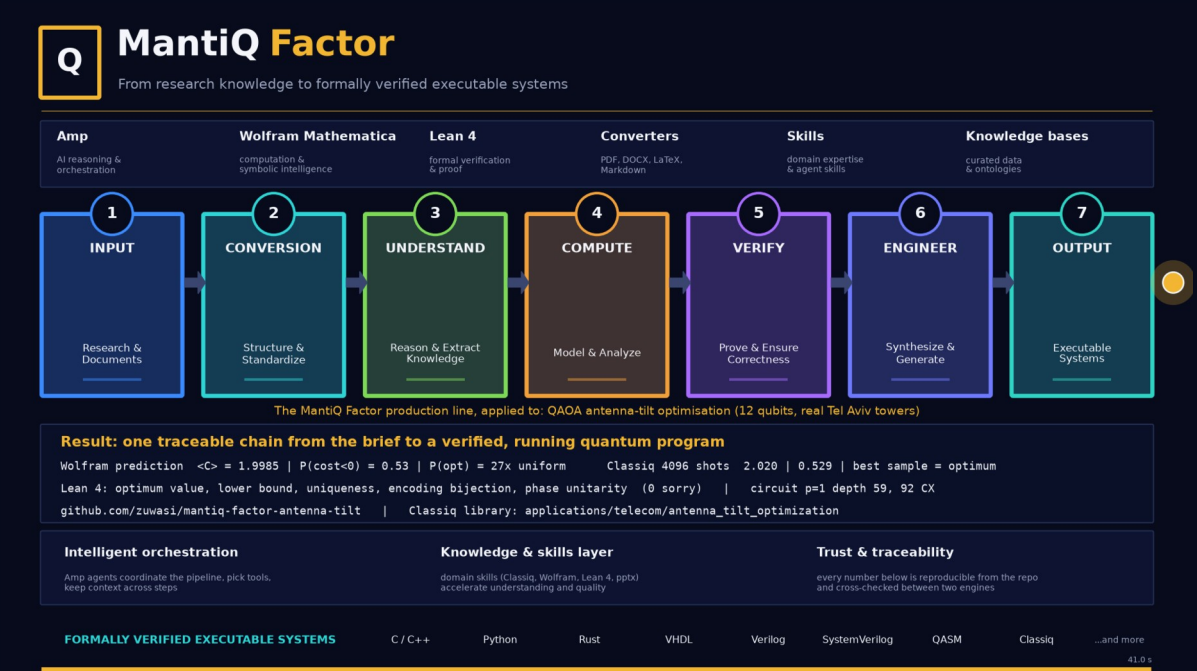
one QAOA layer at 1000 qubits, still 2-local



Team benchmark, 48-region toy network: refinement 5 -> 21 groups. Cycle 3 (13 groups, 13 QAOA jobs) reaches 1.75 = 95% of the per-region ceiling 1.838 (48 jobs); one plan for all gives 1.379.

- ▶ Network -> 5 groups -> 25 -> 125: k-means on region feature vectors, one QAOA job per group, refine where the utility gain is worth it. Our 12-qubit problem is one job in that tree
- ▶ Best non-quantum tools at 1000 qubits: brute force 10²⁸³ s at exascale; exact QUBO solvers run hours to days without certifying; annealing and tabu give a good plan in minutes with no guarantee (1.833 vs 1.838 on the toy benchmark). Nobody can simulate QAOA past about 48 qubits
- ▶ A real 1000-qubit gate-based machine: physical qubits exist (IBM Condor 1121, Atom Computing 1200), but 13,000 CX at 99.9% fidelity means about 13 errors per shot, so the run must be error-corrected (2,000 to 100,000 physical qubits; roadmaps 2028 to 2030)
- ▶ Once available: depth ~10² per layer, 10⁴ shots x 20 iterations = seconds to minutes; the Wolfram hot start cuts the iterations. Same Qmod, Classiq synthesizes for the target; only the backend changes
- ▶ What carries over unchanged: the encoding proof, the 2-locality, the hot-start habit (train angles on a small exact instance, transfer to the large one)

Deliverables: everything is public and reproducible



animation/mantiq_pipeline.mp4 (41 s): the MantiQ Factor pipeline animated with the real numbers of this case, built with matplotlib.

artifact	where
Concluded solution (MIT)	github.com/zuwasi/mantiq-factor-antenna-tilt
Classiq library demo (PR)	applications/telecom/antenna_tilt_optimization in Classiq/classiq-library
Hackathon repo and 5-min video	github.com/zuwasi/quantum-headache-antenna-tilt , youtu.be/OSetTWnRQhc
Data	data/opencellid_il2*.csv (OpenCellID, CC BY-SA 4.0)
Wolfram	wolfram/*.wl, .nb, .pdf, .json (angles, probabilities)
Lean 4	lean/TiltQAOA.lean + build log
Classiq	notebook/*.ipynb, .qmod ; classiq/results/*.json, *.qasm
This deck	presentation/build_deck.js (pptxgenjs) and the PPTX/PDF it builds

What the MantiQ Factor workflow bought us

Compute before you engineer

Wolfram found the angles exactly; the Classiq run confirmed them in one shot and COBYLA had nothing left to do. Blind starts on the simulator got stuck at 35.5 or reached only 7.0 after 20 jobs: shot noise hides the gradient.

Prove the parts that fail silently

Encoding bijection, the true optimum, and the diagonal phase layer are where a QAOA demo can be wrong while still producing plausible histograms.

Make the model decision explicit

The 2-qubit vs 5-qubit choice was argued with numbers (invalid mass, Pauli weight, team gain) and written down before code, so the team could veto it.

Files between stages, not memory

JSON angles, Markdown Hamiltonian, Lean source and Qmod are the interfaces. Any stage can be rerun alone, and this deck was rebuilt from them.

Real data raises the bar cheaply

Six OpenCelliD antennas cost nothing more than a toy grid to model, and the result (only the most overlapping antenna tilts) reads as physics, not as a puzzle.

Agentic orchestration is the glue

Amp drove the converters, Mathematica kernel, Lean build and Classiq SDK from one thread with skills and knowledge bases; humans read numbers and decided.

Thank you

MantiQ Factor: input, conversion, understand, compute, verify, engineer, output

Repository (MIT) github.com/zuwasi/mantiq-factor-antenna-tilt

Classiq library demo [applications/telecom/antenna_tilt_optimization](#)

Hackathon repo github.com/zuwasi/quantum-headache-antenna-tilt

Video youtu.be/OSEtWnRQhc

Data OpenCellID, CC BY-SA 4.0